



ENGINEERING HANDBOOK

Pick — Tome 2

Moteur de recommandation

Projet Pick Your Spotlight

Auteur Chris — Pick Your Spotlight team

Date Juin 2026

Version 1.0

Document confidentiel — usage interne et partenaires · Suite du Tome 1 (Vision & Architecture)

Table des matières

1	Introduction	3
2	Parcours « Pick ce soir »	5
3	Vue d'ensemble du pipeline	8
4	Les entrées du moteur	11
5	Étape par étape	14
6	Filtres successifs	19
7	Paramètres et overrides	22
8	Les trois échelles de score	25
9	Quirks connus	28
10	Modes spéciaux	30
11	Performance	33
12	Debug & observabilité	35
13	Qualité & TNR	38
14	Évolutions prévues	40
15	Glossaire technique produit	42
16	FAQ moteur	44

i À propos de ce document

Ce tome complète le

Tome 1 — Vision & Architecture

en documentant le moteur de recommandation de Pick : parcours utilisateur, pipeline runtime, filtres, scores, debug et qualité. Il s'adresse aux développeurs, QA et partenaires techniques — sans extraits de code ni chemins de fichiers.

1. Introduction

Le **moteur de recommandation** est le cœur technique de Pick. Il transforme un profil de goût cinématographique, enrichi par le contexte du moment, en trois propositions de films — rarement plus — que l'utilisateur peut choisir en quelques secondes. Ce tome documente son fonctionnement produit et technique, en complément du **Tome 1 — Vision & Architecture**.

1.1 — Rôle dans l'écosystème Pick

Dans le Tome 1, le moteur de recommandation apparaît comme l'un des huit modules fonctionnels de Pick. En réalité, il est le *hub central* : l'onboarding l'alimente en signaux, la watchlist et la bibliothèque le nourrissent en retours, les soirées et le mode Duo modifient ses entrées, et les flip cards en sont la vitrine.

Sa mission est double : **filtrer** des centaines de milliers de titres pour ne garder qu'un pool pertinent, puis **sélectionner** les meilleurs candidats avec une couche d'intelligence artificielle qui justifie chaque proposition en langage naturel.

1.2 — Promesse produit

Pick ne propose pas un catalogue. Pick propose une **décision**. Le moteur garantit :

- Trois films adaptés au profil, à l'humeur et aux plateformes disponibles
- Aucun film déjà vu, disliked ou exclu par l'utilisateur
- Des justifications personnalisées (« Pourquoi ce film ce soir ? »)
- Un résultat même en cas de profil incomplet ou de contraintes extrêmes (filet de sécurité)

⚙️ Décision d'architecture

Le pipeline est scindé en deux edge functions : une pour la sélection batch (surprise-personalized) et une pour le scoring fin par film (movie-match). Cette séparation permet un affichage immédiat des propositions LLM pendant que les textes riches se chargent en arrière-plan.

1.3 — Prérequis d'activation

Le pipeline personnalisé complet ne s'active que si l'utilisateur possède au minimum **deux films likés** et un **vecteur de goût 32 dimensions** calculé. En dessous de ce seuil, un fallback TMDB léger propose un titre générique — suffisant pour l'expérience alpha, insuffisant pour la promesse Pick.

3

films affichés typiquement

< 15 s

latence pipeline cible

1.4 — Lien avec le Tome 1

Le Tome 1 posait la vision (« trois films, pas trois cents »), les personas (couple, cinéophile, amis) et l'architecture en couches (React → Supabase → Gemini → TMDB). Ce Tome 2 descend dans le détail du module reco : comment le wizard ou l'écran d'accueil déclenche le pipeline, quelles données entrent, quels filtres s'appliquent, et comment interpréter les métriques de debug.

1.5 — Public et périmètre

Ce document s'adresse aux développeurs rejoignant l'équipe, aux ingénieurs QA responsables des TNR, et aux partenaires techniques évaluant la robustesse du moteur. Il décrit le comportement produit et les mécanismes internes — sans extraits de code ni références aux fichiers sources.

✓ Bonnes pratiques

Avant de modifier le pipeline, lire ce tome et la documentation pipeline associée. Toute modification des filtres, seuils ou fallbacks doit être accompagnée d'un test de non-régression. Ne jamais supposer que deux scores affichés sont comparables sans vérifier leur échelle (voir chapitre 8).

2. Parcours « Pick ce soir »

Deux points d'entrée principaux alimentent le même moteur : l'**écran d'accueil** (tap « Ce soir ») et le **wizard guidé** (parcours /app). Ce chapitre décrit le parcours le plus fréquent — depuis l'accueil.

Parcours « Pick ce soir » — écran d'accueil



Durée cible : moins de 2 minutes - textes enrichis movie-match en arrière-plan

2.1 — Déclenchement

L'utilisateur ouvre Pick depuis son canapé et tape le bouton « Ce soir » sur l'écran d'accueil. Si un profil est prêt (≥ 2 likes, vecteur calculé), le client lance immédiatement la génération. Sinon, un message invite à compléter l'onboarding ou un fallback léger s'affiche.

2.2 — Overlay de chargement

Pendant le pipeline, un overlay semi-transparent masque l'accueil avec une animation de chargement. L'utilisateur voit que Pick « réfléchit » — pas un écran blanc. Dès que les trois propositions LLM arrivent (typiquement 5 à 12 secondes), les flip cards apparaissent sans attendre la fin du movie-match.

2.3 — Flip cards et choix

Chaque flip card présente : affiche TMDB, titre, score de match (%), raison LLM (« Ce thriller psychologique correspond à votre amour récent pour... »). Un swipe horizontal permet de parcourir les trois propositions. Tap sur une card → détail enrichi (synopsis, plateformes, textes movie-match si disponibles).

2.4 — Film du soir

L'utilisateur retient un film comme « film du soir ». Il persiste sur l'accueil via l'overlay TonightPick jusqu'au visionnage ou au lendemain. Ce choix alimente le profil (signal positif différé).

i Note produit

Le wizard /app collecte explicitement type (film/série), humeur, genres et plateformes avant d'invoquer le même pipeline. L'accueil utilise les préférences mémorisées et les overrides voix/ambiance si actifs. Les deux parcours convergent vers la même edge function.

2.5 — Timeline type (accueil)

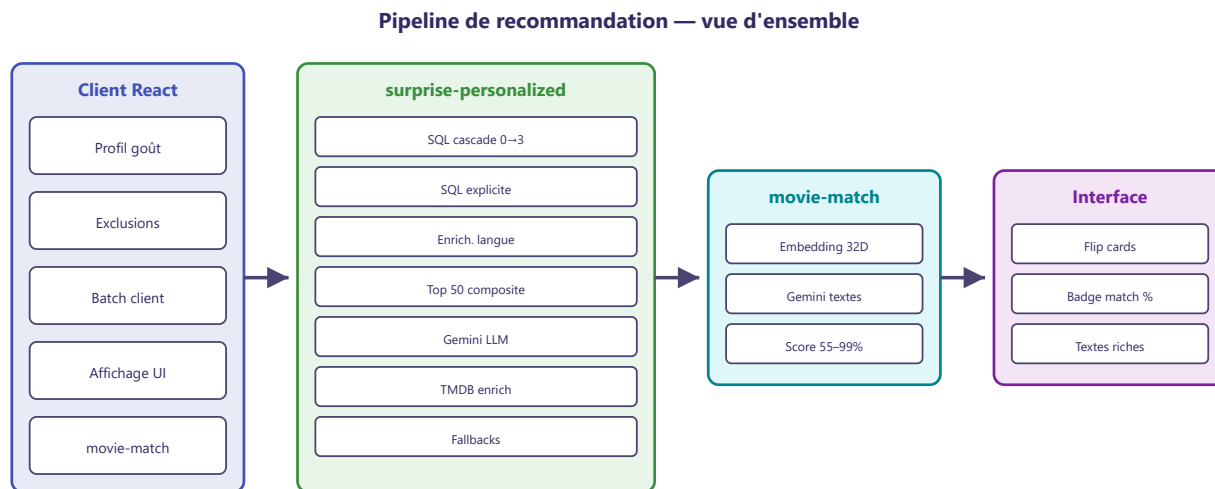
Instant	Action	Perception utilisateur
T+0 s	Tap « Ce soir »	Overlay chargement
T+1 s	Calcul profil + exclusions client	Animation continue
T+3–8 s	SQL cascade + LLM Gemini	Toujours en chargement
T+8–12 s	Batch prêt → flip cards	3 films visibles, raisons LLM
T+12–25 s	movie-match séquentiel (×3)	Textes riches apparaissent progressivement
T+30 s	Choix utilisateur	Film du soir enregistré

✓ **Cas d'usage**

Thomas, 22h un vendredi, tape « Ce soir » sans repasser par le wizard. Pick se souvient de ses plateformes (Netflix, Prime), de son humeur récente (détente) et de ses 47 likes. En 9 secondes, trois comédies françaises récentes apparaissent. Il retourne la deuxième flip card, lit la justification movie-match qui arrive 4 secondes plus tard, et valide « Le Sens de la fête ».

3. Vue d'ensemble du pipeline

Le pipeline de recommandation est un flux orchestré entre le client React, deux edge functions Supabase et les APIs externes TMDb et Gemini. Il se décompose en trois acteurs distincts.



Affichage immédiat après LLM · enrichissement movie-match séquentiel en arrière-plan

3.1 — Couche client

Le client prépare la requête : calcul du profil multi-vecteurs (stable, récent, évitement), agrégation des exclusions (historique, session, duo), résolution des overrides (voix, ambiance, duo). Il invoque surprise-personalized avec le flag debug activé en production sur l'accueil.

À la réception, le batch client extrait les films, affiche immédiatement les trois premiers, puis lance movie-match en séquence pour enrichir textes et scores.

3.2 — Edge surprise-personalized

Cerveau du pipeline. Enchaîne : recherche vectorielle SQL en cascade, complément SQL explicite si besoin, enrichissement langue TMDb, calcul du top 50 composite, sélection Gemini, enrichissement métadonnées TMDb, et chaîne de fallbacks (discover, trending, nuclear, safety net). Retourne un tableau de films avec scores LLM, raisons et métadonnées debug.

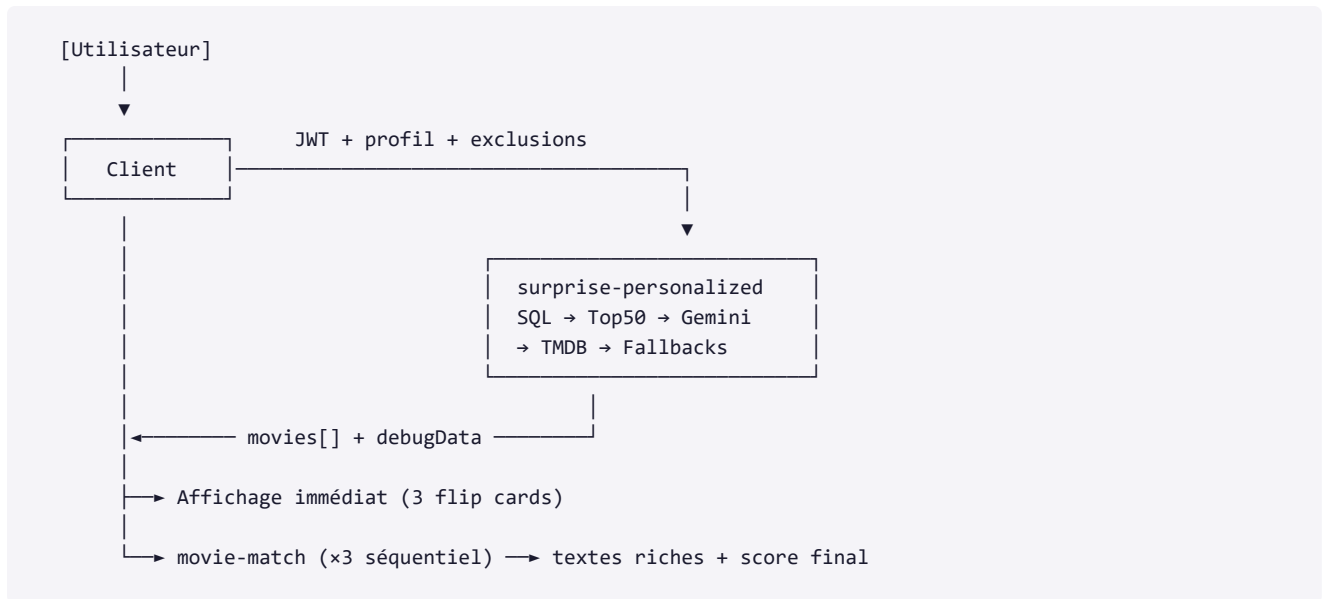
3.3 — Edge movie-match

Second passage IA, par film. Compare l'embedding 32D du film au profil utilisateur, génère headline et texte « pourquoi ça match », produit un score de confiance 55–99 %. Appelé séquentiellement côté client pour limiter la charge Gemini.

⚙️ Décision d'architecture

La plateforme de streaming est filtrée dès la requête SQL et n'est jamais relâchée — même en cascade ou fallback nuclear. C'est une contrainte produit forte : Pick ne recommande que ce que l'utilisateur peut regarder maintenant.

3.4 — Flux de données simplifié



3.5 — Sortie attendue

Typiquement **3 films** visibles (configurable jusqu'à 5 via recommendationCount). Chaque film inclut : identifiant TMDB, titre, affiche, type média, score fusionné, raison LLM, et après movie-match : headline, whyItMatches, providers streaming.

4. Les entrées du moteur

La qualité des recommandations dépend autant des entrées que de l'algorithme. Le moteur consomme une dizaine de paramètres, parfois contradictoires — d'où une hiérarchie de résolution documentée au chapitre 7.

4.1 — Profil et vecteurs de goût

Trois vecteurs 32D complémentaires alimentent la similarité cosinus SQL :

- **Vecteur stable** — goûts long terme, décroissance 150 jours, construit depuis tout l'historique
- **Vecteur récent** — tendances 30 derniers jours, décroissance 21 jours
- **Vecteur évitement** — skips et dislikes, décroissance 60 jours

Le vecteur envoyé au SQL est une combinaison pondérée du stable et du récent, modulée par le niveau d'exploration.

4.2 — Likes et historique

Minimum 2 likes pour activer le pipeline complet. Les likes alimentent les genres préférés (likedGenres) utilisés en SQL niveau 0–2. L'historique complet (vues, skips, dislikes) alimente excludelds — jamais un film déjà interagi ne doit réapparaître (invariant TNR).

4.3 — Plateformes

Liste des provider IDs TMDb (Netflix, Prime, Disney+, etc.) sélectionnés par l'utilisateur. Toujours appliquée en SQL. Si aucune plateforme n'est sélectionnée, le moteur cherche sur l'ensemble du catalogue streaming FR.

4.4 — Exclusions

excludelds agrège : feedback utilisateur, rejets de session en cours, pool chat compagnon, historique partenaire duo, titres déjà proposés ce soir. Côté edge, usedlds reprend excludelds pour éviter les doublons dans les fallbacks (régression corrigée juin 2026).

4.5 — Voix et ambiance

L'utilisateur peut dicter ou saisir une intention (« un polar des années 80 en VO »). La couche voix extrait : genres vocaux (voiceGenres), langue originale, décennie. Ces paramètres override le profil statique avec une priorité élevée (voir chapitre 7).

L'**ambiance** (humeur du wizard ou intent soirée) modifie moodContext et moodBoostGenres — des genres boostés ou pénalisés selon l'humeur sélectionnée (léger, intense, émouvant...).

4.6 — Mode Duo

Quand duoUserIds est présent, le moteur fusionne les profils des deux partenaires : vecteur combiné, exclusions croisées (films déjà vus par l'un ou l'autre), genres communs priorités. Le partenaire n'a pas besoin d'être connecté — son profil est lu côté serveur.

4.7 — Intent Révéler (soirée)

Les soirées groupées propagent des overrides depuis l'événement : genres imposés, humeur, type média. Le pipeline révéler réutilise le même moteur avec ces contraintes figées au moment de la création de la soirée.

Entrée	Source	Impact principal
Vecteur 32D	Profil calculé	Similarité SQL
excludelds	Client + serveur	Films jamais reproposés
platformlds	Préférences utilisateur	Filtre SQL permanent
voiceGenres	Dictée / saisie	Post-filtres + cascade SQL
moodBoostGenres	Wizard / soirée	Boost composite score
duoUserlds	Mode Duo	Fusion profils + exclusions
recommendationCount	Contexte appel	Nombre de films demandés (défaut 5, affichage 3)

⚠ Attention debug

Les paramètres effectifs edge (après résolution voix/ambiance/duo) diffèrent parfois des paramètres client bruts. Toujours vérifier le groupe debug « Paramètres edge » et pipelineStages id 0-edge-request.

5. Étape par étape

Le pipeline complet comporte une quinzaine d'étapes numérotées, traçables via pipelineStages en mode debug. Ce chapitre les détaille dans l'ordre d'exécution.

#	Étape	Comptes typiques	Rôle
0	Préparation client	20–200+ excludelds	Profil, exclusions, invocation edge
1	SQL vectoriel cascade 0→3	Pool cible ~100	Similarité cosinus 32D, relâchement progressif
1.4	SQL explicite	Complète jusqu'à 100	Sans vecteur ou pool insuffisant
1.7	Enrichissement langue	Jusqu'à 60 candidats	Backfill original_language via TMDB
2a–c	Post-filtres	Variable	Langues exclues, genres voix, décennie
2	Top 50 composite	50 films	sim×100 + note TMDB + boost langue
2.2	LLM Gemini	Entrée 50 → sortie count+2	Sélection + raisons JSON
2.5	Retry qualité	+N si < 3 ≥ minScore	Complète depuis pool SQL restant
3	TMDB enrichissement	1 appel / sélection	Métadonnées complètes, providers
4	discover-fallback	0–3 films	Trending TMDB avec plateformes
4b	trending-fallback	0–3 films	Popular / trending élargi
4c	nuclear-fallback	0–3 films	Genres et note levés, plateforme conservée
5	Final safety net	Garantit count films	Tri origine + filet genre/langue
6	movie-match client	1 appel / film séquentiel	Textes riches + score final

5.1 — Étape 0 : préparation client

Avant tout appel réseau, le client calcule le profil multi-vecteurs, normalise les plateformes, agrège excludelds depuis feedback Supabase, sessionStorage et partenaire duo. Il envoie recommendationCount (souvent 5 côté edge, 3 affichés), explorationLevel, et debug:true sur l'accueil.

5.2 — Étape 1 : cascade SQL vectorielle

Cascade SQL vectorielle — relâchement progressif



La plateforme n'est JAMAIS levée en cascade SQL

Quatre niveaux de relâchement, du plus strict au plus permissif. Le moteur tente le niveau 0 ; si le pool reste sous ~100 candidats non-interagis, il passe au niveau suivant. Le flag `tasteCascadeTriggered` indique qu'un relâchement a eu lieu.

5.3 — Étape 1.4 : SQL explicite

Si le vecteur est absent ou le pool vectoriel insuffisant, une requête sans similarité complète le pool via `match_movies_explicit` en trois niveaux (goût strict → plateforme seule).

5.4 — Étape 2.2 : sélection Gemini

Le top 50 composite est numéroté et envoyé à Gemini avec le profil utilisateur complet. Le modèle retourne un JSON `selections[{{rank, matchScore, reason}}]`. En cas d'échec LLM, un fallback déterministe prend les N premiers du top 50 avec `reason:null`.

⚙ Décision d'architecture

Le LLM reçoit `count+2` sélections (ex. 5 si `count=3`) pour laisser une marge au client qui n'affichera que les 3 meilleurs après fusion et exclusions résiduelles.

5.5 — Fallbacks discover, trending, nuclear

Si SQL + LLM produisent zéro film valide, la chaîne de fallbacks TMDb s'enclenche : discover avec filtres plateforme, puis trending/popular, puis mode nuclear (genres et note minimum levés, plateforme toujours active). Chaque fallback est tracé dans `fallbackTrace`.

5.6 — Safety net et backfill client

Le filet de sécurité edge (étape 5-final-safety) garantit le nombre de films demandés en complétant depuis le pool SQL restant, avec vérification genres/langues exclus. Côté client, `ensureRecommendationBatch` peut backfiller de 1 à 3 films si le batch initial est incomplet — sans appel TMDb aveugle (correction audit juin 2026).

⚠ Attention debug

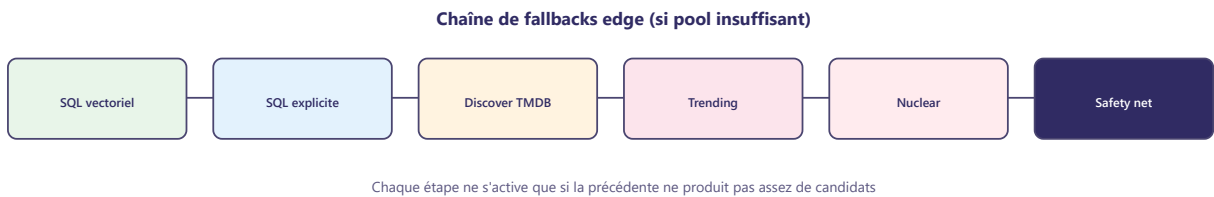
Un batch de 1 seul film indique généralement des contraintes trop strictes (plateforme rare + genres voix étroits) ou un pool SQL épuisé. Vérifier `sqlCascadeLevel`, `fallbackTrace` et le nombre d'`excludelds`.

5.7 — Constantes serveur clés

Constante	Valeur	Signification
TARGET	100	Pool SQL cible
llmPoolSize	50	Films envoyés au LLM
BATCH	500	Candidats par round RPC
minMatchScore	~60	Seuil retry qualité étape 2.5
min_rating SQL	6 (niv. 0–2), 0 (niv. 3)	Note TMDB minimum

6. Filtres successifs

Le moteur applique une série de filtres en entonnoir : chaque étape réduit le pool candidat. Comprendre l'ordre et les conditions de relâchement est essentiel pour diagnostiquer les recommandations trop génériques ou trop restrictives.



6.1 — Filtres SQL (edge)

Filtre	Quand appliqué	Relâché à
Plateforme	Toujours (si sélectionnée)	Jamais
Langue originale	SQL niv. 0 + voix	Niveau 1+
Décennie (min/max year)	SQL niv. 0 + voiceDecade	Niveau 1+ (sauf voiceDecade hard)
Genres profil (likedGenres)	SQL niv. 0–1	Niveau 2 (voiceGenres conservés)
Note minimum TMDB	SQL niv. 0–2 (≥ 6)	Niveau 3 (min_rating=0)
Popularité minimum	SQL niv. 0–2	Niveau 3
excludelds	Toujours	Jamais
Genres exclus profil	Toujours	Jamais (safety net final)

6.2 — Post-filtres (après SQL, avant LLM)

Trois post-filtres affinent le pool avant le top 50 :

- **2a — Langues exclues** : retire les original_language dans effectiveExcludedLangs
- **2b — Genres voix** : si ≥ 5 candidats restent après filtre voiceGenres, applique le filtre ; sinon conserve le pool (évite le vide)
- **2c — Décennie** : voiceDecade en mode hard (élimination stricte) ou profileDecades en mode soft (boost seulement)

6.3 — Filtres LLM et TMDB

Gemini sélectionne dans le top 50 en tenant compte du profil textuel — ce n'est pas un filtre binaire mais une re-ranking intelligente. L'enrichissement TMDb valide que chaque ID existe, récupère providers streaming et corrige le media_type (movie vs tv).

6.4 — Chaîne de fallbacks

Fallback	Déclencheur	Ce qui est relâché	Ce qui est conservé
SQL explicite	Pool vectoriel < TARGET	Similarité vectorielle	Plateforme, exclusions
LLM déterministe	Gemini KO / JSON invalide	Sélection IA	Top 50 trié composite
Retry qualité 2.5	< 3 films ≥ minMatchScore	Seuil score LLM	Pool SQL restant
discover-fallback	0 films post-LLM	Personnalisation goût	Plateforme, excludeds
trending-fallback	discover insuffisant	Filtres discover	Plateforme
nuclear-fallback	trending insuffisant	Genres, note min	Plateforme uniquement
Safety net final	count < demandé	Ordre origine	Genres/langues exclus
Backfill client	Batch < 3 affichés	—	Films réserve SP non affichés

✓ **Bonnes pratiques**

Lors d'un diagnostic « recommandations génériques », vérifier tasteCascadeTriggered et le niveau SQL atteint. Un niveau 3 systématique signale un profil contraint ou un pool d'exclusions trop large — pas un bug LLM.

7. Paramètres et overrides

De multiples sources alimentent le moteur simultanément. Sans hiérarchie claire, les conflits produiraient des comportements imprévisibles. Pick applique une résolution strictement ordonnée.

7.1 — Hiérarchie de priorité

Priorité décroissante :

1. DUO – profils fusionnés

2. VOIX – intention explicite

3. AMBIANCE – humeur / soirée

4. PROFIL – goûts statiques

En mode Duo, les exclusions des deux partenaires s'additionnent et le vecteur fusionné prime. Une commande vocale « je veux un western » override les genres profil mais pas les plateformes duo communes. L'ambiance « intense » booste thrillers et dramas même si le profil stable penche comédie.

7.2 — Paramètres ajustables clés

Paramètre	Défaut	Effet
recommendationCount	5 (edge) / 3 (UI)	Nombre de films produits vs affichés
explorationLevel	0–2	Poids vecteur récent vs stable
llmPoolSize	50	Taille du pool pré-LLM
minMatchScore	~60	Seuil qualité retry 2.5
min_rating	6 → 0	Note TMDB min par niveau cascade
voiceGenreThreshold	5	Min candidats avant filtre genre voix
preferredLangBoost	+15	Bonus langue préférée composite
spMinValid (fusion)	60	Score SP minimum avant fusion MM

7.3 — Overrides voix

voiceGenres, voiceOriginalLanguage et voiceDecade sont extraits de la dictée utilisateur. voiceDecade peut être « hard » (filtre éliminatoire post-SQL) ou absent (profil décennies en soft boost). Les genres voix remplacent likedGenres à partir du niveau SQL 2.

7.4 — Overrides ambiance et soirée

moodContext décrit l'humeur en texte libre pour le prompt LLM. moodBoostGenres liste les genres à favoriser (ex. « Comédie » en mode détente). En intent Révéler, mediaType, genres et mood sont figés depuis l'événement soirée.

⚙️ **Décision d'architecture**

Les overrides ne modifient jamais excludelds ni platformlds — ce sont des contraintes dures. Seuls les critères de « goût positif » (genres, décennie, langue) sont sujets à la hiérarchie.

7.5 — explorationLevel

Trois niveaux modulant le mélange stable/récent : niveau 0 (conservateur, profil stable dominant), niveau 1 (équilibré), niveau 2 (explorateur, vecteur récent fortement pondéré). Impacte le query_vector SQL et le ton du prompt LLM.

8. Les trois échelles de score

Pick affiche un pourcentage de match sur chaque flip card. Derrière ce chiffre unique se cachent **trois systèmes de scoring distincts**, non directement comparables. Les confondre est l'erreur de debug la plus fréquente.



8.1 — Sim% (similarité SQL)

Produit de la similarité cosinus entre le vecteur utilisateur et l'embedding film, multiplié par 100. Plage typique 0–100. Sert au tri du pool SQL et aux tables debug sqlCandidates. Ne tient pas compte de l'humeur du moment ni du contexte LLM.

8.2 — Score composite (pré-LLM)

Formule : Sim% + note TMDb (vote_average) + bonus +15 si langue originale préférée. Utilisé pour construire le top 50. Échelle non bornée supérieurement (~0–115). N'apparaît pas directement dans l'UI.

8.3 — LLM matchScore

Score assigné par Gemini dans selections[], borné 60–99 par le prompt. Reflète l'adéquation contextuelle (« ce soir, comédie légère »). Affiché immédiatement à onBatchReady avant movie-match.

8.4 — movie-match score

Score 55–99 produit par la edge function movie-match, combinant embedding 32D et jugement Gemini. Alimente le badge final et les textes headline / whyItMatches. Seuil plancher profil à 55 côté MM.

8.5 — Règles de fusion client

Le score affiché final résulte de mergeRecommendationScores :

1. Ignorer tout score SP (LLM) < 60 % — considéré aberrant
2. Si SP valide et MM disponibles : prendre le maximum des deux
3. Sinon : SP valide seul, ou MM seul, ou null

⚠ Attention debug

Ne jamais trier les films finaux par Sim% en supposant refléter l'ordre UI. Un film Sim% 78 peut être classé derrière un Sim% 65 si le LLM l'a favorisé contextuellement. Utiliser llmSelections puis le score fusionné client.

8.6 — Pourquoi trois échelles ?

Chaque échelle optimise un objectif différent : Sim% pour la proximité vectorielle brute, composite pour diversifier avec la qualité TMDb, LLM pour le contexte du soir, movie-match pour la personnalisation fine et les textes. Les fusionner en une seule métrique trop tôt dégraderait l'un de ces objectifs.

Échelle	Moment	Visible UI	Comparable avec
Sim%	SQL	Debug seulement	Composite (base)
Composite	Pré-LLM	Non	Sim% (composante)
LLM matchScore	Post-Gemini	Teaser immédiat	MM (après fusion)
movie-match	Post-MM	Badge final	LLM (après fusion)

9. Quirks connus

L'audit de juin 2026 a identifié plusieurs comportements non intuitifs du moteur. Ils ne sont pas des bugs mais des conséquences de compromis architecturaux — à connaître pour éviter fausses alertes.

9.1 — reason: null

Symptôme : colonne « Raison » vide dans les tables debug, ou justification absente sur une flip card.

Causes : fallback LLM déterministe (Gemini KO), retry qualité étape 2.5, parsing JSON partiel (regex fallback sans champ reason).

Impact UI : l'interface conserve parfois le teaser LLM initial même si reason devient null côté edge. Les textes movie-match (headline, whyItMatches) compensent généralement l'absence.

Debug : vérifier llmSelections, fallbackTriggered dans pipelineStages 2.2, et logs ⚠️ FALLBACK côté client.

9.2 — Trois échelles non comparables

Symptôme : film A Sim% 82 classé derrière film B Sim% 71 dans le résultat final.

Cause : le LLM et movie-match re-rangent selon le contexte, pas la similarité brute.

Impact : confusion lors de l'analyse debug, fausses régressions signalées.

Debug : comparer llmSelections.matchScore et score MM final, pas sqlCandidates.similarity.

9.3 — movie-match FALLBACK

Symptôme : log client « ⚠️ FALLBACK » pour un film, textes génériques, score MM absent ou par défaut.

Cause : Gemini indisponible ou timeout dans movie-match. Retour objet { fallback: true }.

Impact : retry automatique après 4 secondes ; film de réserve waterfall si échec persistant.

Debug : chronologie [Pick🕒] movie-match · vérifier charge Gemini · un seul FALLBACK occasionnel est normal en pic.

9.4 — Autres quirks documentés

Quirk	Impact	Tip debug
tasteCascadeTriggered	Reco plus générique	Noter sqlCascadeLevel atteint
Batch 1 film	UX dégradée	excludelds count + plateformes rares
Plateforme bypass LLM	Filtre plateforme souvent redondant post-SQL	platformPool vs llmFiltered

Quirk	Impact	Tip debug
usedIds régression 2026-06-03	Doublons en fallback	Corrigé — must inclure excludelds
Backfill TMDB aveugle	Films hors profil	Corrigé juin 2026 — réserve SP uniquement

i Corrections récentes (v1.23)

Renommage debug top20→top50, sql50→sqlCandidates. platformFallbackTriggered remplacé par tasteCascadeTriggered. platformCandidatesCount n'est plus écrasé par la taille du pool LLM.

10. Modes spéciaux

Le pipeline s'adapte à des contextes d'appel différents. Le cœur reste identique ; seules les entrées et la sortie diffèrent.

10.1 — Fallback < 2 likes

Sans profil minimum, surprise-personalized n'est pas invoquée. Le client appelle getSurpriseRecommendation (TMDB trending/discover léger) pour afficher un titre unique non personnalisé. Message UX encourageant l'onboarding.

10.2 — Vecteur null

Profil avec likes mais vecteur non calculé (cache expiré, migration en cours) : SQL vectoriel sauté, SQL explicite puis discover-fallback. Résultats moins pertinents mais non vides.

10.3 — Mode Duo

duoUserIds active la fusion de profils : vecteur combiné (moyenne pondérée), excludeIds union des deux historiques, partnerUserId passé au RPC SQL. Genres communs favorisés dans le prompt LLM.

10.4 — Intent Révéler (soirée)

runRevealPipeline propage mood, genres et mediaType depuis l'événement. Overlay révélation sans flash accueil. Pas de movie-match client sur la création d'événement (count:3 direct). Synchronisation présence temps réel pour la révélation collective.

10.5 — Wizard vs accueil

Aspect	Wizard /app	Accueil « Ce soir »
Collecte contexte	Explicite (étapes)	Mémorisé + voix/ambiance
debug: true	Non par défaut	Oui (PICK-DEBUG prod)
Edge function	surprise-personalized	Idem
movie-match	Oui, séquentiel	Oui, séquentiel
UI résultats	ResultScreen plein écran	TonightPickOverlay

10.6 — Création événement soirée

Lors de la création d'une soirée (CreateEventPage), le pipeline produit 3 films sans enrichissement movie-match côté client — les métadonnées SP suffisent pour la prévisualisation organisateur. Le movie-match intervient au moment de la révélation si configuré.

✓ **Bonnes pratiques**

Tester chaque mode spécial indépendamment : profil neuf (<2 likes), duo actif, soirée avec intent restrictif, wizard avec genres étroits. Les TNR phase 2 couvrent les fixtures JSON pour SP complet et safety net 1 film.

11. Performance

La promesse Pick (« moins de 2 minutes ») impose une latence pipeline cible inférieure à 15 secondes pour l’affichage des flip cards. Ce chapitre documente les timings observés, les goulots d’étranglement et la feuille de route optimisation.

11.1 — Timings typiques (accueil)

Étape	Durée typique	Variable principale
Profil client	100–300 ms	Cache user_taste_vectors
SQL cascade	800 ms – 2 s	Niveau atteint, excludelds count
Enrichissement langue	0–3 s	Candidats lang null
Top 50 + post-filtres	< 100 ms	Taille pool
Gemini LLM	2–6 s	Charge API Google
TMDB enrich batch	500 ms – 1.5 s	Nombre sélections
Fallbacks (si déclenchés)	+1–4 s	Type fallback
movie-match ×3	6–15 s total	Séquentiel — 2–5 s/appel

Total perçu utilisateur : 5–12 s jusqu’aux flip cards (sans movie-match), 15–25 s jusqu’aux textes riches complets.

11.2 — Goulot principal : movie-match séquentiel

Chaque appel movie-match enchaîne embedding lookup, prompt Gemini et formatage réponse. Trois films affichés = trois appels séquentiels (~2–5 s chacun). C’est le poste d’amélioration prioritaire (voir chapitre 14).

⚙️ Décision d’architecture

L’affichage immédiat post-LLM (onBatchReady) compense la latence movie-match : l’utilisateur voit des propositions en ~10 s pendant que les textes riches se chargent. Ne pas bloquer l’UI sur movie-match.

11.3 — Autres facteurs de latence

- **Cascade SQL niveau 3** — plusieurs rounds RPC 500 candidats
- **Enrichissement langue 1.7** — jusqu’à 60 appels TMDB unitaires
- **excludelds volumineux** — 200+ IDs ralentissent les RPC PostgreSQL
- **Fallback chain** — discover + trending + nuclear en cascade

11.4 — Roadmap optimisation (P2)

Optimisation	Gain estimé	Statut
movie-match parallèle (×3 concurrent)	– 10 s	Planifié
Cache embeddings TMDB	– 1–2 s SQL	Partiel (movie_embeddings)
recentTasteVector in SQL	Meilleure pertinence sans latence	Planifié
Timeouts UX + retry	Perception	Toast erreur fait (1.4)
engineMeta → monitoring	Observabilité prod	Planifié (2.8)

✓ **Bonnes pratiques**

Utiliser les préfixes [Pick🕒] et engineMeta.timings pour profiler chaque requête. Comparer SQL ms vs LLM ms vs MM ms avant d'optimiser — ne pas paralléliser MM si le goulot est SQL niveau 3.

12. Debug & observabilité

Le moteur expose une instrumentation riche activée par `debug:true`. En production sur l'accueil, les groupes [PICK-DEBUG] structurent la console développeur pour tracer chaque étape sans outil externe.

12.1 — Groupes console [PICK-DEBUG]

Groupe	Contenu
 Paramètres par étape	pipelineStages — synthèse entrée/sortie/fallback
 Paramètres envoyés	Vecteur, plateformes, genres, excludeIds (client)
 1 SQL vectoriel 32D	sqlCandidates, sqlCascadeLevel, sqlRpcParams, snippet RPC
 Détail par niveau	sqlLevelDebug — films gagnés par niveau 0–3
 1 + SQL explicite	explicitFallbackDebug
 Profil → LLM	llmProfile, systemPrompt complet
 2 Top N → LLM	top50 — jusqu'à 50 films numérotés
 3 Sélections LLM	llmSelections — matchScore + reason
 Films fallback	fallbackTrace — discover/trending/nuclear
 3.5 TMDB enrich	tmdbEnrichment — OK/échec par ID
 4 Films finaux edge	finalMoviesList avant movie-match
 Timings pipeline	engineMeta.timings par étape
 4 Résultat final MM	Score MM, rich texts vs FALLBACK

12.2 — pipelineStages (v25+)

Tableau ordonné d'objets { id, name, params, fallbackTriggered, fallbackReason, inputCount, outputCount }. Permet de reconstruire le parcours complet sans lire chaque groupe console.

id	Étape tracée
0-client-request	Paramètres avant invoke
0-edge-request	Paramètres effectifs (voix/mood/duo résolus)

id	Étape tracée
1-sql-vector	Cascade SQL 0→3
1.4-sql-explicit	Complément explicite
1.7-lang-enrich	Backfill langue TMDB
2a-c post-filters	Langue, genre voix, décennie
2-top50-composite	Top 50 score composé
2.2-llm-selection	Gemini + fallback déterministe
2.5-quality-retry	Complément minMatchScore
3-tmdb-enrich	Lookup TMDB batch
4-discover / 4b / 4c	Chaîne fallbacks TMDB
5-final-safety	Filet sécurité edge
6-movie-match	Appels séquentiels client

12.3 — engineMeta

Objet retourné avec la réponse SP : `tasteCascadeTriggered`, `sqlCandidatesCount`, `llmPoolCount`, `mode` (normal/fallback), `timings` détaillés (`sqlMs`, `langMs`, `llmMs`, `tmdbMs`, `fallbackMs`). Alimente l'optimisation continue et le monitoring futur (Sentry, tâche 2.8).

⚠ Attention debug

`debugData` n'est pas exposé en production aux utilisateurs finaux — visible console uniquement. Ne jamais logger `systemPrompt` ou profil complet côté serveur persistant (RGPD). Le flag `debug` reste actif sur l'accueil pour l'équipe alpha.

13. Qualité & TNR

Le moteur de recommandation est protégé par une suite de tests de non-régression (TNR) en trois phases. L'objectif : toute modification du pipeline doit être vérifiable sans regarder 500 films.

13.1 — État juin 2026

131	70
tests unitaires passants	tests pipeline dédiés

Suite	Tests	Périmètre
taste-engine	28	Vecteurs, decay, profil multi-vecteurs
recommendation-pipeline	23	Fonctions pures SP extraites (composite, cascade, parse LLM)
recommendation-batch	9	Extraction payload, fusion scores, normalisation textes
recommendation-batch-integration	8	Mocks SP/MM, backfill, genres exclus
recommendation-non-regression	15	Invariants exclusions, usedIds, filet 1→3

13.2 — Phases TNR pipeline

Phase	Statut	Livrables
Phase 1 — Unitaires	✅ Done	taste-engine, batch helpers, fonctions pures SP
Phase 2 — Intégration	✅ Done (25/06)	Fixtures JSON SP/MM, batch-integration, non-regression étendu
Phase 3 — E2E	🕒 En cours	pipeline.spec stable, smoke post-deploy, couverture 70 %

13.3 — Invariants non négociables

- **Jamais de film déjà interagi** — `excludelds` respecté en sortie edge et client
- **Genres exclus absents** — `safety net checkFinalSafety`
- **usedIds inclut excludelds** — régression 2026-06-03 documentée
- **Backfill sans TMDB aveugle** — réserve SP uniquement (correction audit)
- **Plateforme conservée en fallback nuclear** — contrainte produit
- **mergeScore ignore SP < 60** — pas de score aberrant affiché

13.4 — Gaps phase 3

- E2E pipeline avec credentials stables (pas seulement mocks)
- Checklist manuelle post-deploy § smoke tests 3.2
- Mesure automatisée couverture 70 % sur logique métier lib/
- Tests duo bout-en-bout non automatisés

✓ **Bonnes pratiques**

Avant tout refactor HomeScreen ou pipeline : `npm run test:unit` (131 tests, ~10 s). Ajouter un test pour chaque bug corrigé. Les fixtures `sp-response-full.json` et `sp-response-safety-net-1.json` servent de références shape API.

14. Évolutions prévues

Le moteur alpha est fonctionnel mais pas optimisé. Cette section recense les améliorations planifiées, classées par impact.

14.1 — Performance

- **movie-match parallèle** — Promise.all sur 3 films au lieu de séquentiel (~10 s typique)
- **recentTasteVector in SQL** — intégrer le vecteur récent directement dans la RPC pour meilleure pertinence sans latence client
- **Batch TMDB lang enrich** — réduire les appels unitaires étape 1.7

14.2 — Maintenabilité

- **HomeScreen refactor (2.5)** — extraction hooks useProfileLoader, useRecommendationEngine, useTonightMovieState — petits diffs, pas de gros bang
- **TonightPickContext (1.20)** — réduire ~30 props overlay, PR par étapes pair humain
- **Split ResultScreen + Overlay (2.10)** — composants >1000 lignes chacun

14.3 — Observabilité & prod

- **engineMeta** → **Sentry/dashboard (2.8)** — timings, cascade level, fallback rate
- **Métriques produit** — time-to-pick, satisfaction post-reco, taux fallback
- **Secret TMDB serveur (1.16)** — avant beta testeurs externes

⚙ Décision d'architecture

Aucun refactor pipeline profond sans tests 1.19 en place — désormais satisfait (131 tests). Les prochains refactors HomeScreen sont débloqués mais doivent rester incrémentaux.

15. Glossaire technique produit

Termes spécifiques au moteur de recommandation Pick, complémentaires au glossaire produit du Tome 1.

Terme	Définition
Vecteur stable	Embedding 32D goûts long terme, decay 150 jours
Vecteur récent	Embedding 32D tendances 30 jours, decay 21 jours
Vecteur évitement	Embedding 32D skips/dislikes, decay 60 jours
Cascade goût (SQL)	4 niveaux 0→3 relâchant progressivement lang, année, genres, note
SQL explicite	Requête sans similarité vectorielle, complément pool
Top 50 composite	Pool pré-LLM trié par $\text{sim} \times 100 + \text{note} + \text{boost langue}$
Sim%	Similarité cosinus $\times 100$ — échelle debug SQL
matchScore LLM	Score 60–99 assigné par Gemini à la sélection
surprise-personalized	Edge function pipeline batch principal
movie-match	Edge function scoring fin + textes riches par film
Filet sécurité (safety net)	Mécanisme edge garantissant le count demandé
Pool backfill	Films réserve SP utilisés par le client si batch < 3
excludIds	Liste TMDB IDs jamais reproposés (historique + session)
usedIds	Set edge incluant excludIds — anti-doublon fallbacks
tasteCascadeTriggered	Flag : relâchement SQL niveau > 0
onBatchReady	Callback client — affichage immédiat post-LLM
pipelineStages	Tableau debug trace ordonnée des étapes
engineMeta	Métriques timings et counts retournées par SP
PICK-DEBUG	Préfixe groupes console debug pipeline
Fallback nuclear	Dernier recours TMDB — plateforme seule, genres/note levés

Terme	Définition
TNR	Tests de Non-Régression — 131 tests unitaires juin 2026

16. FAQ moteur

Pourquoi seulement 1 film parfois ?

Contraintes trop strictes combinées : plateforme rare (ex. Canal+ seul), genres voix étroits, excludelds volumineux (utilisateur actif), cascade niveau 3 insuffisante. Le filet sécurité edge et le backfill client tentent d'atteindre 3 — si 1 seul persiste, vérifier debug fallbackTrace et sqlCandidatesCount.

Pourquoi une cascade SQL et pas un seul filtre ?

Un filtre unique strict produirait souvent zéro résultat (profils exigeants, plateformes limitées). La cascade préserve la qualité quand possible (niveau 0) et dégrade gracieusement (niveaux 1–3) avant d'en appeler aux fallbacks TMDb. La plateforme reste non négociable à tous les niveaux.

Pourquoi reason vide (null) ?

Gemini n'a pas répondu ou le JSON est partiel. Le fallback déterministe sélectionne par score composite sans générer de raison. L'UI peut conserver un teaser antérieur ; movie-match compense via whyItMatches. Vérifier llmSelections et fallbackTriggered.

Pourquoi le score affiché diffère du Sim% debug ?

Trois échelles distinctes (chapitre 8). Le badge UI montre $\max(\text{SP} \geq 60, \text{MM})$, pas Sim%. C'est voulu — le contexte du soir prime sur la similarité brute.

Pourquoi movie-match prend-il autant de temps ?

Appels séquentiels (3× Gemini). Optimisation parallèle planifiée. L'UI n'attend pas — flip cards visibles dès le LLM.

Le wizard et l'accueil utilisent-ils le même moteur ?

Oui — même edge function surprise-personalized. Différences : collecte contexte (explicite vs mémorisé), debug activé sur accueil seulement, UI résultats (ResultScreen vs Overlay).

Comment tester sans API live ?

npm run test:unit — 131 tests dont 70 pipeline, fixtures JSON, zéro réseau. E2E pipeline.spec.ts mocke les edge functions.

i Aller plus loin

Documentation pipeline détaillée (diagrammes Mermaid, mapping debug complet) · Backlog TNR phases 1–3 · Smoke tests § 3.2 pour validation manuelle post-deploy.



Fin du Tome 2

Moteur de recommandation

Pick Your Spotlight — Engineering Handbook
Version 1.0 — Juin 2026

Tome 1 : Vision, Architecture & Principes
Tome 3 (à venir) : Pick Together, Soirées & Social